

Glossary

This glossary provides a comprehensive reference for the technical terms and configurations used throughout the project, organized by their functional domains.

JavaScript & TypeScript Basics

These terms form the core language foundation of the project's codebase.

- **const**: Declares a variable that cannot be reassigned, used for values that should remain constant.
 - **let**: Declares a variable that can be reassigned later when a value needs to change.
 - **var**: An old-style variable declaration that should be avoided in favor of **const** or **let**.
 - **async**: Marks a function as asynchronous, allowing it to return a **Promise** and use **await**.
 - **await**: Pauses an **async** function until a **Promise** resolves; only valid inside **async** functions.
 - **Promise**: Represents a value or error that will be available in the future, such as after a database call.
 - **=> (Arrow Function)**: Shorthand syntax for writing functions.
 - **export / import**: **export** makes code available to other files, while **import** brings it into the current file.
 - **Default Export**: The primary export of a file, imported without using curly braces.
 - **type / interface**: TypeScript features used to describe the shape of data to catch mistakes and provide autocomplete.
-

Environment & Configuration

Settings and variables that define how and where the application runs.

- **Environment (env)**: The specific context where the app runs, such as "development" on a local machine or "production" on Vercel.
 - **Environment Variable**: A setting read from the environment (e.g., `DATABASE_URL`) that is not committed to Git.
 - **.env**: A local file used to hold environment variables for development; typically listed in `.gitignore`.
 - **process.env**: The Node.js object that holds all environment variables.
-

Authentication & Security

Terms related to securing user data and managing access.

- **JWT (JSON Web Token)**: A signed string encoding user information like IDs or roles.

- **JWT_SECRET:** A secret string used to sign and verify tokens; it must remain private and consistent across the server.
 - **Authorization Header:** An HTTP header used to send tokens to the server (e.g., Authorization: Bearer <jwt>).
 - **Hashing:** A one-way transformation of data, such as a password, so the plain text is never stored.
 - **bcrypt:** A specific library used for safely hashing and checking passwords during login.
-

API & HTTP

The communication protocols between the frontend and backend.

- **API:** The Application Programming Interface consisting of backend routes that the frontend calls.
 - **Endpoint:** A specific API URL combined with an HTTP method (e.g., POST /api/auth/register).
 - **REST:** An architectural style for APIs using standard HTTP methods like GET, POST, PUT, and DELETE.
 - **Request / Response:** The "Request" is what the client sends to the server, and the "Response" is what the server sends back.
 - **Status Code:** A number indicating the result of a request (e.g., 200 OK, 401 Unauthorized, 404 Not Found).
 - **Preflight:** An **OPTIONS** request sent by the browser before certain cross-origin requests to check CORS rules.
 - **CORS (Cross-Origin Resource Sharing):** Rules allowing a frontend on one domain to call an API on a different domain.
-

Database (Prisma)

Terms involving the management and structure of project data.

- **Prisma:** An ORM (Object-Relational Mapper) that allows database interaction using TypeScript instead of raw SQL.
 - **Schema:** The definition of data models, columns, and relations, found in prisma/schema.prisma.
 - **Migration:** A recorded change to the database structure, such as adding a new table or column.
 - **Seed:** A script used to populate the database with initial or demo data.
 - **DATABASE_URL:** The connection string used to link the app to the PostgreSQL database.
-

Backend (Hono / Node)

Specific concepts related to the server-side logic.

- **Middleware:** Code that runs for nearly every request to handle tasks like auth or logging before the main handler.
 - **Route / Handler:** A "Route" maps a path to a "Handler," which is the function that returns the response.
 - **Context (c):** The Hono object passed to handlers that holds the request, headers, and response helpers.
 - **Serverless:** A deployment model where code runs in short-lived functions (like Vercel) rather than a permanent server.
-

Frontend (React / Vite)

Core concepts for building the user interface.

- **Component:** A reusable piece of the UI, such as a button or an entire page.
 - **Hook:** A React function (starting with use) that adds state or side effects to a component.
 - **State:** Data that, when updated, triggers the UI to re-render.
 - **Props:** Data passed from a parent component down to a child component.
 - **Build:** The process of optimized source code into production-ready files in the dist/ folder.
-

Deployment & Other Terms

Services used to host the app and general development tools.

- **Vercel / Cloudflare Pages:** Vercel hosts the API, while Cloudflare Pages hosts the frontend static site.
- **Neon:** The serverless PostgreSQL service used as the project's database.
- **Origin:** The specific URL (scheme + host + port) used to define CORS permissions.
- **ALLOWED_ORIGINS:** A list of frontend URLs permitted to call the backend API.
- **npm / npx:** npm installs packages, while npx runs packages without permanent installation.
- **Dependency:** An external package the project relies on, listed in package.json.
- **Singleton:** A design pattern where a single shared instance (like the Prisma client) is reused to save resources.

Tech Stack & Deployment

Layer	Technology	Purpose
Frontend	React 18	Industry-standard library for building declarative UIs.
Language	TypeScript	Improves maintainability by catching bugs at build time.
Build Tool	Vite	Fast ESM-based dev server and build tool.
Backend API	Hono	Lightweight framework with an Express-like API.
ORM	Prisma	Provides type-safe database access and schema management.
Database	PostgreSQL	Relational database hosted via Neon, Vercel, or Supabase.
State	Zustand	Manages global state (cart, auth) with minimal boilerplate.
Styling	Tailwind CSS	Utility-first framework for rapid UI building.
Auth	JWT + bcryptjs	Stateless authentication and secure password hashing.

Deployment Strategy

The project utilizes a hybrid cloud approach:

- **Backend (API) & Database:** Deployed on **Vercel**. The API runs as serverless functions in the `api/` directory.
- **Frontend:** Deployed on **Cloudflare Pages**.

- **Configuration:** The frontend must have the VITE_API_ORIGIN environment variable set to the Vercel API URL to communicate correctly.

DevOps Tools

- **Wrangler:** Used to deploy the frontend dist/ folder to Cloudflare Pages.
- **npm scripts:** Key commands include dev (frontend), build (compilation), and pages:deploy.

Folder Structure

File/Folder	Purpose
api/	Vercel serverless entry point; handles all /api/* requests.
dist/	Compiled frontend build output deployed to Cloudflare Pages.
node_modules/	Installed project dependencies.
prisma/	Database schema, migrations, and seeding scripts.
public/	Static assets served as-is (e.g., favicon).
scripts/	Utility and deployment scripts.
server/	Backend application code including Hono app, routes, and middleware.
src/	Frontend React source code.

File/Folder	Purpose
.wrangler/	Cloudflare Wrangler local development cache.
vercel.json	Configuration for Vercel: framework, headers (CORS), and env.

Key Subdirectories

- **api/**: Contains the catch-all serverless handler ([[...path]].ts) that forwards requests to the Hono app.
- **server/**:
 - **app-vercel.ts**: Main Hono entry for Vercel; handles CORS and mounts routes.
 - **routes/**: Contains modules for auth, products, orders, and admin.
 - **middleware/**: Includes JWT verification logic (e.g., auth.ts).
- **src/**:
 - **components/**: Divided into layout/ (Navbar, Sidebar), ui/ (primitives like buttons), and specific features like AuthModal.
 - **lib/**: Contains the API client (api.ts) and global state store (store.ts).
 - **pages/**: Includes route-specific components for the homepage, product pages, and the Admin dashboard.